



# CODE SPORT

This month, as is our practice for every year-end column, we will discuss a bunch of programming questions.

Last month, we started discussing software bloat and its impact on the efficiency of our software. Every year, in the CodeSport column for December, I feature a bunch of programming questions that I have discussed in the column over the past 12 months. So let's do the same this year and continue our discussion of software bloat next month.

- (1) You are given two sorted lists of size  $X$  and  $Y$ . How would you determine the  $k^{\text{th}}$  smallest element in the union of the two lists? What would be the complexity of your algorithm?
- (2) You are given an array  $X$  of  $N$  integers. You are asked to write an algorithm which returns the 'hot element' (one that appears more than  $N/3$  times), if one exists. If no hot element exists, your algorithm should be able to determine that. The other constraint is that you can only compare the elements of the array for equality, i.e., you can only use comparisons of the form ' $X[i] = X[j]$ ', which take constant time. What is the complexity of your algorithm?
- (3) You are given three containers of capacities 10 litres, 7 litres and 4 litres each. The 7 and 4 litre containers are full of water. The 10 litre container is empty. The only type of action allowed is pouring the contents of one container into another, and you can stop pouring only when either (a) the source container is completely empty; or (b) the receiving container is entirely full. Can you write an algorithm to determine whether there exists a sequence of actions that leaves exactly 2 litres in the 7 litre container?
- (4) Given a binary tree  $T = (V, E)$  represented in adjacency list format, along with a root node ' $r$ ', you are asked to design an algorithm such that it can pre-process the tree, and answer a bunch of queries of the form, "*Is vertex ' $u$ ' an ancestor of vertex ' $v$ ' ?*" in constant time. Note that only the query response needs to be in constant time, whereas you can spend time pre-processing the tree to build sufficient information to answer the queries in constant time. Note that a vertex ' $u$ ' is said to be an ancestor of vertex ' $v$ ' if the path from root node ' $r$ ' to vertex ' $v$ ' passes through vertex ' $u$ '.
- (5) You are given a directed graph  $G(V, E)$  in the form of an adjacency list. Give a linear-time algorithm for determining whether the graph contains an odd-length cycle. If you can show that it does not contain an odd-length cycle, can you infer anything about whether it is a bi-partite graph or not?
- (6) You are given a directed acyclic graph  $G(V, E)$ . A Hamiltonian path is a path in a directed graph that touches each vertex exactly once. Write an algorithm to determine whether the directed graph contains a Hamiltonian path? What is the complexity of your algorithm? Instead of being given a DAG, if you are asked to